Mardi 17/11, après-midi — Les objets

Kevin Bardoux

mardi 17 novembre 2026

Table des matières

Objectifs 2
Après-midi — Les objets 2
Les objets (20 min) 2
Les objets, en image 3
Les objets, l'exemple minimal 3
Les objets — attention et vérification 4
Une valeur par défaut : `??` (2 min) 4
Pas à pas ensemble — La fiche d'un musée (35 min) 5
Variante à faire seuls 8
Enregistrer et envoyer (2 min) 8
Exercices — Les objets 8
Palier 1 — Le nom complet, l'étiquette d'un musée (25 min) . . 8
Palier 2 — Les majeurs, nettoyer un musée (45 min) 9
Palier 3 — Nettoyer tout le fichier, des valeurs par défaut (45 min) 10
Enregistrer et envoyer (2 min) 11
À retenir 11
Si le temps le permet 11
Débrief 11
Sources 11

Note

Compétences visées : C4 · résolution de problème **Livrable** : le pas à pas `pas-a-pas-objets` envoyé sur GitHub, et `node --test m1-nom-complet.test.js` au vert.

! Le minimum du jour

- Le pas à pas « La fiche d'un musée », fait avec le groupe.
- Le palier 1 : `node --test m1-nom-complet.test.js` et `node --test o1-etiquette-musee.test.js` au vert.

Les paliers 2 et 3, puis la rubrique « Si le temps le permet », sont la suite naturelle, pour ceux qui avancent.

Objectifs

- Créer un objet, lire et modifier ses propriétés.
- Reconnaître `undefined` quand une propriété n'existe pas.
- Utiliser `map` et `filter` sur un tableau d'objets.

Comment se passe l'après-midi. Une seule notion : les objets. Les données réelles sont presque toujours des tableaux d'objets : un objet par musée, par produit, par personne. Vous réutilisez `map` et `filter` sur ces tableaux.

Après-midi — Les objets

Les objets (20 min)

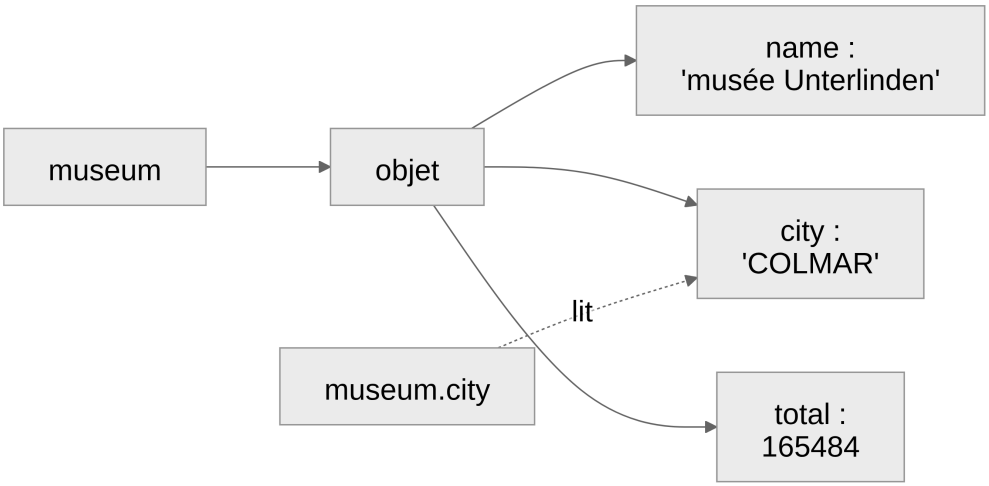
- **Pourquoi** : garder ensemble les informations d'une même chose.
- **Objet** : des propriétés nommées, `clé: valeur`.
- **Lire** : `objet.clé` ; une clé absente donne `undefined`.

Pourquoi. Un musée a un nom, une ville, un nombre de visites. Trois variables séparées se perdent vite, surtout pour cent musées. Un objet les garde ensemble, sous un seul nom.

Définition. Un objet regroupe des propriétés nommées, écrites `clé: valeur` entre accolades ; on lit une propriété avec un point :

`museum.name` (MDN Web Docs, s. d.-b). De vraies données, c'est le plus souvent un **tableau d'objets** : un objet par musée.

Les objets, en image



Un objet : une étiquette, museum, sur un ensemble de propriétés clé: valeur.

Le texte du schéma : l'étiquette `museum` désigne un objet ; l'objet a trois propriétés, `name`, `city` et `total`, chacune avec sa valeur ; `museum.city` lit la valeur `'COLMAR'`.

Les objets, l'exemple minimal

Le premier programme de la notion, annoté :

```
const museum = {
  name: 'musée Unterlinden',
  city: 'COLMAR',
  total: 165484,
};

console.log(museum.name); // 'musée Unterlinden'
museum.total = 170000;
console.log(museum.country); // undefined
```

Ligne 1

Les accolades créent un objet ; l'étiquette `museum` le désigne.

Ligne 2

Une propriété : la clé `name`, deux-points, la valeur, puis une virgule.

Ligne 7

Le point lit la valeur d'une propriété : `objet.clé`.

Ligne 8

Une affectation change la valeur ; une clé qui n'existait pas serait ajoutée.

Ligne 9

Une clé qui n'existe pas : aucune erreur, la valeur `undefined`.

Les objets — attention et vérification

> ⚠ Attention
>
> `museum.Name` (majuscule) donne `undefined`, sans message d'erreur : les noms de propriétés sont sensibles à la casse. Un `undefined` inattendu ? Vérifiez d'abord l'orthographe de la clé.

- **Vérifiez** : comment lire la ville de `museum` ?
- Que donne `museum.Total` ?

Une valeur par défaut : `??` (2 min)

> ⚠ Attention
>
> Une propriété **absente** donne `undefined` ; dans les vraies données, une valeur **inconnue** vaut souvent `null`. Pour donner une valeur de secours, utilisez `??` : `a ?? b` vaut `a`, sauf si `a` vaut `null` ou `undefined` ; alors il vaut `b` (MDN Web Docs, s. d.-a).
>
> ```
> const museum = { name: 'musée Unterlinden', total: null };
>
> console.log(museum.city ?? 'ville inconnue'); // 'ville inconnue'
> console.log(museum.total ?? 0);               // 0
> ```
>
> N'utilisez pas `||` pour cela : il remplacerait aussi un vrai `0` (un musée sans visite) ou un texte vide.

`??` sert dans un exercice de palier 3 (`o2-valeurs-par-defaut`), et jeudi, pour un total inconnu (`dc4-total-visites`).

Pas à pas ensemble — La fiche d'un musée (35 min)

- Le formateur code, vous tapez en même temps, une étape à la fois.
- Code et commentaires : `//` au-dessus de chaque ligne.
- Dossier : `pas-a-pas-objets`, fichier `fiche.js` (vide).
- Perdu ? Dossier `rattrapage/objets/`, une étape à la fois.

Palier 1 : guidé. Ouvrez `pas-a-pas-objets/fiche.js` : il ne contient que trois lignes de commentaire. Le musée de l'exemple et ses visites viennent du jeu « Fréquentation des Musées de France » (Ministère de la Culture, s. d.). Dans le terminal : `cd pas-a-pas-objets`.

Étape 1 — Créer l'objet. Dans le fichier vide, créez l'objet `museum`, avec ses trois propriétés.

Avant d'exécuter : que va afficher la console : l'objet, ou seulement son nom ?

```
// Objet : les informations d'un même musée
const museum = {
  // Propriété : la clé name, puis sa valeur
  name: 'musée Unterlinden',
  // Propriété : la ville
  city: 'COLMAR',
  // Propriété : les visites de 2019
  total: 165484,
};
// Affiche tout l'objet
console.log(museum);
```

Enregistrez, puis lancez `node fiche.js` dans le terminal, depuis `pas-a-pas-objets`.

Vous devez voir, à la fin de la sortie :

```
{ name: 'musée Unterlinden', city: 'COLMAR', total:
165484 }
```

Perdu ? Copiez `rattrapage/objets/etape-1/fiche.js` à la place de votre `fiche.js`, puis repartez avec le groupe.

Étape 2 — Lire des propriétés. Lisez le nom, la ville, puis une propriété qui n'existe pas.

Avant d'exécuter : `museum.country` n'existe pas : erreur, ou autre chose ?

```
// Le point lit une propriété : objet.clé
console.log(museum.name);
// La ville du musée
console.log(museum.city);
// Une clé qui n'existe pas donne undefined
console.log(museum.country);
```

Enregistrez, puis lancez `node fiche.js` dans le terminal, depuis `pas-a-pas-objets`.

Vous devez voir, à la fin de la sortie :

```
musée Unterlinden
COLMAR
undefined
```

Perdu ? Copiez `rattrapage/objets/etape-2/fiche.js` à la place de votre `fiche.js`, puis repartez avec le groupe.

Étape 3 — Modifier, ajouter. Changez le nombre de visites, ajoutez le département, puis affichez l'objet.

Avant d'exécuter : combien de propriétés l'objet aura-t-il ?

```
// Affectation : change la valeur d'une propriété
museum.total = 170000;
// Nouvelle clé : la propriété est ajoutée
museum.department = 'Haut-Rhin';
// Affiche tout l'objet
console.log(museum);
```

Enregistrez, puis lancez `node fiche.js` dans le terminal, depuis `pas-a-pas-objets`.

Vous devez voir, à la fin de la sortie :

```
{
  name: 'musée Unterlinden',
  city: 'COLMAR',
  total: 170000,
  department: 'Haut-Rhin'
}
```

Perdu ? Copiez `rattrapage/objets/etape-3/fiche.js` à la place de votre `fiche.js`, puis repartez avec le groupe.

Étape 4 — Un tableau d'objets. Créez un tableau de trois musées, puis récupérez leurs noms avec `map`.

Avant d'exécuter : `map` va-t-il renvoyer des objets, ou des textes ?

```
// Tableau d'objets : trois musées
const museums = [
  // Chaque élément est un objet
  { name: 'musée Arthur Rimbaud', total: 29212 },
  { name: "musée de l'Ardenne", total: 51775 },
  { name: 'musée du château-fort', total: 59853 },
];
// map : le nom de chaque musée
console.log(museums.map((item) => item.name));
```

Enregistrez, puis lancez `node fiche.js` dans le terminal, depuis `pas-a-pas-objets`.

Vous devez voir, à la fin de la sortie :

```
[
  'musée Arthur Rimbaud',
  "musée de l'Ardenne",
  'musée du château-fort'
]
```

Perdu ? Copiez `rattrapage/objets/etape-4/fiche.js` à la place de votre `fiche.js`, puis repartez avec le groupe.

Variante à faire seuls
Pour ceux qui ont validé l'étape avant les autres, dans le même fichier. Avec `find` (ce matin), cherchez dans `museums` le premier musée de plus de 50 000 visites. Affichez-le. **Réussi si** : la console affiche `{ name: "musée de l'Ardenne", total: 51775 }`.

Enregistrer et envoyer (2 min)
Depuis `~/dwwm/dwwm-tp/semaine-07` : `git status`, `git add pas-a-pas-objets`, `git commit -m "Ajoute le pas à pas objets"`, puis `git push`. **Réussi si** : `git status` affiche `rien à valider, la copie de travail est propre`.

Exercices — Les objets

Palier 1 — Le nom complet, l'étiquette d'un musée (25 min)

1. `m1-nom-complet.js` : lire deux propriétés.
2. `o1-etiquette-musee.js` : `'nom (VILLE)'`.
3. **Réussi si** : leurs tests au vert.

Palier 1 : guidé. Durée : 25 min.

1. Ouvrez `m1-nom-complet.js`. `fullName(person)` reçoit un objet `{ firstName, lastName }`.
2. Renvoyez le prénom, une espace, puis le nom : `person.firstName + ' ' + person.lastName`, ou un gabarit `` `${person.firstName} ${person.lastName}` ``.
3. Lancez `node --test m1-nom-complet.test.js`.
4. Ouvrez `o1-etiquette-musee.js`. `museumLabel(museum)` renvoie le nom, une espace, puis la ville entre parenthèses.
5. Lancez `node --test o1-etiquette-musee.test.js`. Puis un commit par exercice réussi.

Réussi si : les deux commandes affichent `i fail 0`.

> **i** Aides
>
> - **Le cours** : Mardi après-midi, « Les objets » ; le pas à pas, étape 2.
> - **L'IA** : CS50.ai (palier IA 1) : demandez comment lire une propriété d'un objet, pas le code de l'exercice.
> - **MDN** : Utiliser les objets.

Palier 2 — Les majeurs, nettoyer un musée (45 min)

1. `m2-majeurs.js` : `filter`, puis `map`, sur des objets.
2. `dc1`, puis `dc2` : nettoyer une ligne réelle.
3. **Réussi si** : leurs trois fichiers de tests au vert.

Palier 2 : recommandé, semi-guidé. Durée : 45 min.

1. **`m2-majeurs`** : `adultNames(people)` renvoie les **noms** des personnes de 18 ans ou plus. Gardez d'abord les majeurs (`filter`), puis prenez leur nom (`map`). `node --test m2-majeurs.test.js`.
2. **Les vraies données.** Le fichier `data/musees-grand-est-2019.json` est un tableau d'objets : la fréquentation des musées du Grand-Est en 2019 (Ministère de la Culture, s. d.), sous Licence Ouverte (Etalab, s. d.). Lisez `data/LISEZMOI.md` : les nombres y sont écrits **en texte** (`"1449"`), et certains sont vides.
3. **`dc1-nettoyer-nombre`** : `toNumber(value)` renvoie le nombre, ou `null` si le texte est vide ou n'est pas un nombre.
   - `Number('1449')` donne `1449` ; un texte vide doit donner `null`.
   - Étape documentation : dans MDN, trouvez comment savoir si un résultat est `NaN` (« pas un nombre ») : page *Number*.
   - `node --test dc1-nettoyer-nombre.test.js`.
4. **`dc2-nettoyer-musee`** : `cleanMuseum(raw)` transforme une ligne brute en objet propre, avec des clés en anglais. La consigne liste chaque clé ; `toNumber` est déjà importée. `node --test dc2-nettoyer-musee.test.js`.

Réussi si : `node --test m2-majeurs.test.js`, `node --test dc1-nettoyer-nombre.test.js` et `node --test dc2-nettoyer-musee.test.js` affichent `ℹ fail 0`.

> ℹ Aides
>
> - **Le cours** : Mardi après-midi, « Les objets » ; Mardi matin, « filter et find ».
> - **L'IA** : CS50.ai (palier IA 1) : demandez pourquoi `Number('abc')` donne `NaN`, pas le code.
> - **MDN** : Number ; String.prototype.trim().

Palier 3 — Nettoyer tout le fichier, des valeurs par défaut (45 min)

- `dc3-nettoyer-tout.js` : 116 lignes, 114 musées propres.
- `o2-valeurs-par-defaut.js` : `??`, sans perdre un vrai `0`.
- **Réussi si** : leurs tests `.defi` au vert.

Palier 3 : autonome. Durée : 45 min.

1. **`dc3-nettoyer-tout`** : `cleanAll(rawList)` nettoie tout le fichier, en écartant les lignes sans identifiant et en ne gardant qu'une fois un musée présent deux fois. Il reste 114 musées sur 116 lignes. `node --test dc3-nettoyer-tout.defi.test.js`.
2. **`o2-valeurs-par-defaut`** : `withDefaults(museum)` renvoie un nouvel objet où la ville manquante devient `'inconnue'` et le total inconnu, `0`, sans remplacer un vrai `0` ni un texte vide. `node --test o2-valeurs-par-defaut.defi.test.js`.

Réussi si : les deux commandes affichent `ℹ fail 0`.

> ℹ Autonome
>
> Vous allez bloquer : c'est voulu, c'est le métier. Méthode : 1. lisez le message d'erreur en entier ; 2. cherchez-le dans la fiche « Lire un message d'erreur » ; 3. lisez la documentation ; 4. après 15 minutes bloqué sur le même point, demandez.

> **Aides**
>
> - **Le cours** : l'encadré « Une valeur par défaut : `??` » ; `data/LISEZMOI.md`.
> - **L'IA** : CS50.ai (palier IA 1) : demandez comment retenir les identifiants déjà vus, pas le code.
> - **MDN** : Opérateur de coalescence des nuls (??) ; Array.prototype.includes().

Enregistrer et envoyer (2 min)

Même rituel : `git status`, `git add`, `git commit -m "…"`, `git push`, depuis `semaine-07`. C'est la fin de la demi-journée : le `git push` est obligatoire.

À retenir

> **À retenir**
>
> - **Objet** : `{ clé: valeur }` ; on lit `objet.clé`.
> - **Clé absente** : `undefined`, sans erreur ; la casse compte.
> - **Tableau d'objets** : `map` et `filter`, avec `item.clé` dans la fonction.

Si le temps le permet

- Les paliers 2 et 3 non terminés.
- Le parcours d'autonomie, lignes 4 et 6.

Débrief

- À quoi voit-on qu'une clé est mal orthographiée ?
- Pourquoi les nombres du fichier sont-ils en texte ?

Sources

Etalab. s. d. « Licence Ouverte / Open Licence ». Consulté le 26 septembre 2026. https://www.etalab.gouv.fr/wp-content/uploads/2014/05/Licence_Ouverte.pdf.

MDN Web Docs. s. d.-a. « Opérateur de coalescence des nuls (??) ». Consulté le 1 octobre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Operators/Nullish_coalescing.

MDN Web Docs. s. d.-b. « Utiliser les objets - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Guide/Working_with_objects.

Ministère de la Culture. s. d. « Fréquentation des Musées de France (data.gouv.fr) ». Consulté le 26 septembre 2026. https://www.data.gouv.fr/datasets/frequentation-des-musees-de-france-1.